***BRACT's***

**Vishwakarma Institute of Information Technology, Pune Practical Implementation Sheet**

Department: CS-AI Semester: 5 Academic Year: 2026-27

Division: E Batch: 2 Practical No:9 Roll no: 21 PRN No: 12411682

**Subject Name: Deep Learning**

**Name of Student: Pranay Zunzunwala**

**Problem Statement:** Design and implement deep learning application for audio classification that incorporates Speech to Text (STT) and Text to Speech (TTS) using appropriate deep learning architecture and evaluate its suitable performance using suitable metrics

## Algorithm:

Step 1: Import the required libraries such as TensorFlow, Keras, Librosa, NumPy, Pandas, Matplotlib, Seaborn, Transformers, gTTS, and Scikit-learn.

Step 2: Load the Mini Speech Commands dataset containing audio recordings of eight spoken commands such as yes, no, up, down, left, right, on, and off.

Step 3: Preprocess the audio recordings by converting them to mono, resampling them to 16 kHz, and padding or truncating them to a fixed duration.

Step 4: Extract audio features using log-Mel spectrograms to represent the frequency characteristics of the speech signals.

Step 5: Split the dataset into training, validation, and testing subsets for model development and performance evaluation.

Step 6: Build a CNN model using convolutional layers, batch normalization, ReLU activation, pooling layers, dropout, and fully connected layers for audio classification.

Step 7: Configure the CNN training parameters, including learning rate, batch size, optimizer, loss function, and number of epochs, and train the model using the training dataset.

Step 8: Evaluate the CNN model on the validation dataset and monitor its training performance using accuracy and loss curves.

Step 9: Integrate a pretrained Whisper-tiny English model from Hugging Face Transformers to perform Speech-to-Text (STT) and convert spoken audio into textual transcripts.

Step 10: Integrate Text-to-Speech (TTS) using gTTS to convert text, including the predicted audio command, into spoken output.

Step 11: Evaluate the audio classification model on the test dataset using accuracy, precision, recall, and F1-score.

Step 12: Generate a confusion matrix and classification report to analyze the model's class-wise performance and identify misclassified audio commands.

Step 13: Upload or record a sample audio file and process it through the STT system and the trained CNN model to obtain the speech transcript, predicted command, and classification confidence.

Step 14: Display the top predicted audio classes, compare the actual and predicted commands where ground-truth labels are available, and analyze the advantages and limitations of CNN-based audio classification, STT, and TTS.

Step 15: Save the trained CNN model, class-label mapping, and evaluation results for future use and deployment.

**Code:**

```
## 1. Install dependencies
"""
# Commented out IPython magic to ensure Python compatibility.
 %pip -q install librosa soundfile scikit-learn matplotlib seaborn pandas transformers accelerate gTTS

"""## 2. Imports and setup"""
import os, random, zipfile, urllib.request, json, warnings
from pathlib import Path

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
import librosa
import soundfile as sf
import tensorflow as tf

from sklearn.model_selection import train_test_split
from sklearn.preprocessing import LabelEncoder
from sklearn.metrics import (
    accuracy_score, classification_report, confusion_matrix,
    precision_recall_fscore_support
)

SEED = 42
random.seed(SEED)
np.random.seed(SEED)
```

```
tf.random.set_seed(SEED)

print("TensorFlow:", tf.__version__)
print("GPU devices:", tf.config.list_physical_devices("GPU"))

"""## 3. Download the Mini Speech Commands dataset"""

DATA_ROOT = Path("/content") if Path("/content").exists() else Path.cwd()
DATA_DIR = DATA_ROOT / "mini_speech_commands"
ZIP_PATH = DATA_ROOT / "mini_speech_commands.zip"
DATA_URL =
"https://storage.googleapis.com/download.tensorflow.org/data/mini_speech_commands.zip"

if not DATA_DIR.exists():
    print("Downloading dataset...")
    urllib.request.urlretrieve(DATA_URL, ZIP_PATH)
    print("Extracting dataset...")
    with zipfile.ZipFile(ZIP_PATH, "r") as zf:
        zf.extractall(DATA_ROOT)

if not DATA_DIR.exists():
    raise FileNotFoundError(f"Dataset folder was not created: {DATA_DIR}")

class_names = sorted(p.name for p in DATA_DIR.iterdir() if p.is_dir())
if not class_names:
    raise RuntimeError("No class folders found in the dataset.")

print("Dataset:", DATA_DIR)
print("Classes:", class_names)
for cls in class_names:
    print(f"{cls:>10}: {len(list((DATA_DIR / cls).glob('*.wav')))} files")

"""## 4. Explore the dataset"""

counts = {c: len(list((DATA_DIR / c).glob("*.wav"))) for c in class_names}
plt.figure(figsize=(10, 4))
plt.bar(counts.keys(), counts.values())
plt.title("Audio clips per class")
plt.xlabel("Command")
plt.ylabel("Number of clips")
plt.xticks(rotation=35)
plt.tight_layout()
plt.show()

example_path = next((DATA_DIR / class_names[0]).glob("*.wav"))
example_audio, example_sr = sf.read(example_path)
```

```
if example_audio.ndim > 1:
    example_audio = example_audio.mean(axis=1)
print("Example:", example_path.name, "| sample rate:", example_sr)
plt.figure(figsize=(10, 2))
plt.plot(np.arange(len(example_audio)) / example_sr, example_audio)
plt.title(f"Waveform: {class_names[0]}")
plt.xlabel("Time (seconds)")
plt.tight_layout()
plt.show()

"""## 5. Convert audio into log-Mel spectrograms"""

SAMPLE_RATE = 16000
DURATION = 1.0
NUM_SAMPLES = int(SAMPLE_RATE * DURATION)
N_MELS = 64
N_FFT = 512
HOP_LENGTH = 256
MAX_PER_CLASS = 1200  # Set to None to use all available files.

def load_audio_mono(path):
    y, sr = sf.read(path)
    if y.ndim > 1:
        y = y.mean(axis=1)
    y = y.astype(np.float32)
    if sr != SAMPLE_RATE:
        y = librosa.resample(y, orig_sr=sr, target_sr=SAMPLE_RATE)
    if len(y) < NUM_SAMPLES:
        y = np.pad(y, (0, NUM_SAMPLES - len(y)))
    else:
        y = y[:NUM_SAMPLES]
    return y.astype(np.float32)

def audio_to_logmel(y):
    mel = librosa.feature.melspectrogram(
        y=y, sr=SAMPLE_RATE, n_fft=N_FFT, hop_length=HOP_LENGTH,
        n_mels=N_MELS, fmin=50, fmax=7600
    )
    logmel = librosa.power_to_db(mel, ref=np.max)
    logmel = (logmel - logmel.mean()) / (logmel.std() + 1e-6)
    return logmel.astype(np.float32)

X_list, labels_list, paths_list = [], [], []
for cls in class_names:
    files = list((DATA_DIR / cls).glob("*.wav"))
    random.shuffle(files)
```

```
    if MAX_PER_CLASS is not None:
        files = files[:MAX_PER_CLASS]
    for wav_path in files:
        try:
            waveform = load_audio_mono(wav_path)
            X_list.append(audio_to_logmel(waveform))
            labels_list.append(cls)
            paths_list.append(str(wav_path))
        except Exception as e:
            print("Skipping", wav_path, ":", e)

X = np.asarray(X_list, dtype=np.float32)[..., np.newaxis]
label_encoder = LabelEncoder()
y = label_encoder.fit_transform(np.asarray(labels_list))
NUM_CLASSES = len(label_encoder.classes_)

print("Feature shape:", X.shape)
print("Classes:", list(label_encoder.classes_))

"""## 6. Split training, validation, and test data"""

X_train, X_temp, y_train, y_temp = train_test_split(
    X, y, test_size=0.30, random_state=SEED, stratify=y
)
X_val, X_test, y_val, y_test = train_test_split(
    X_temp, y_temp, test_size=0.50, random_state=SEED, stratify=y_temp
)
print("Training:", X_train.shape, "Validation:", X_val.shape, "Test:", X_test.shape)

"""## 7. Build the CNN"""

model = tf.keras.Sequential([
    tf.keras.layers.Input(shape=X_train.shape[1:]),
    tf.keras.layers.Conv2D(32, 3, padding="same", activation="relu"),
    tf.keras.layers.BatchNormalization(),
    tf.keras.layers.MaxPooling2D(2),
    tf.keras.layers.Conv2D(64, 3, padding="same", activation="relu"),
    tf.keras.layers.BatchNormalization(),
    tf.keras.layers.MaxPooling2D(2),
    tf.keras.layers.Conv2D(128, 3, padding="same", activation="relu"),
    tf.keras.layers.BatchNormalization(),
    tf.keras.layers.MaxPooling2D(2),
    tf.keras.layers.Dropout(0.30),
    tf.keras.layers.GlobalAveragePooling2D(),
    tf.keras.layers.Dense(128, activation="relu"),
    tf.keras.layers.Dropout(0.30),
```

```
    tf.keras.layers.Dense(NUM_CLASSES, activation="softmax")
])

model.compile(
    optimizer=tf.keras.optimizers.Adam(learning_rate=1e-3),
    loss="sparse_categorical_crossentropy",
    metrics=["accuracy"]
)
model.summary()

"""## 8. Train the CNN"""

callbacks = [
    tf.keras.callbacks.EarlyStopping(
        monitor="val_loss", patience=5, restore_best_weights=True
    ),
    tf.keras.callbacks.ReduceLROnPlateau(
        monitor="val_loss", factor=0.5, patience=2, min_lr=1e-6
    ),
    tf.keras.callbacks.ModelCheckpoint(
        "best_audio_cnn.keras", monitor="val_loss", save_best_only=True
    )
]

history = model.fit(
    X_train, y_train,
    validation_data=(X_val, y_val),
    epochs=25,
    batch_size=64,
    callbacks=callbacks,
    verbose=1
)

"""## 9. Plot training curves"""

fig, axes = plt.subplots(1, 2, figsize=(12, 4))
axes[0].plot(history.history["accuracy"], label="Training")
axes[0].plot(history.history["val_accuracy"], label="Validation")
axes[0].set_title("Accuracy")
axes[0].set_xlabel("Epoch")
axes[0].legend()
axes[1].plot(history.history["loss"], label="Training")
axes[1].plot(history.history["val_loss"], label="Validation")
axes[1].set_title("Loss")
axes[1].set_xlabel("Epoch")
axes[1].legend()
```

```
plt.tight_layout()
plt.show()

"""## 10. Evaluate the CNN"""

test_probs = model.predict(X_test, batch_size=64, verbose=0)
y_pred = np.argmax(test_probs, axis=1)

test_accuracy = accuracy_score(y_test, y_pred)
precision, recall, f1, _ = precision_recall_fscore_support(
    y_test, y_pred, average="weighted", zero_division=0
)
print(f"Test accuracy      : {test_accuracy:.4f}")
print(f"Weighted precision : {precision:.4f}")
print(f"Weighted recall    : {recall:.4f}")
print(f"Weighted F1-score  : {f1:.4f}")
print("\nClassification report:")
print(classification_report(
    y_test, y_pred, target_names=label_encoder.classes_, zero_division=0
))

cm = confusion_matrix(y_test, y_pred)
plt.figure(figsize=(9, 7))
sns.heatmap(cm, annot=True, fmt="d", cmap="Blues",
            xticklabels=label_encoder.classes_,
            yticklabels=label_encoder.classes_)
plt.xlabel("Predicted")
plt.ylabel("Actual")
plt.title("CNN Confusion Matrix")
plt.tight_layout()
plt.show()

"""## 11. Save the model and labels"""

model.save("audio_command_cnn.keras")
with open("label_classes.json", "w") as f:
    json.dump(label_encoder.classes_.tolist(), f)
print("Saved audio_command_cnn.keras and label_classes.json")

"""## 12. Initialize Whisper STT

The warnings about deprecated `forced_decoder_ids` or generation settings are usually non-fatal.
This cell suppresses only those known deprecation warnings; real exceptions will still be displayed. Whisper downloads model files on the first run.
"""
```

```
import torch
from transformers import pipeline

warnings.filterwarnings(
    "ignore",
    message=".*forced_decoder_ids.*",
    category=UserWarning
)
warnings.filterwarnings(
    "ignore",
    message=".*generation_config.*",
    category=UserWarning
)

stt_device = 0 if torch.cuda.is_available() else -1
try:
    stt_pipe = pipeline(
        "automatic-speech-recognition",
        model="openai/whisper-tiny.en",
        device=stt_device
    )
    print("Whisper initialized on", "GPU" if stt_device == 0 else "CPU")
except Exception as e:
    stt_pipe = None
    print("Whisper initialization failed:", repr(e))
    print("The CNN classification part can still be used.")

"""## 13. Initialize TTS"""

from gtts import gTTS
from IPython.display import Audio, display

def speak_text(text, lang="en"):
    text = str(text).strip()
    if not text:
        print("No text supplied to TTS.")
        return
    output_path = "tts_output.mp3"
    gTTS(text=text, lang=lang).save(output_path)
    display(Audio(filename=output_path, autoplay=False))
    return output_path

print("TTS function ready. gTTS requires an internet connection.")

"""## 14. Upload an audio file"""
```

```
from google.colab import files

uploaded = files.upload()
if not uploaded:
    raise ValueError("No file uploaded. Please upload a WAV or MP3 file.")

uploaded_audio_path = next(iter(uploaded.keys()))
if not os.path.exists(uploaded_audio_path):
    raise FileNotFoundError(f"Uploaded file is not available: {uploaded_audio_path}")

print("Uploaded:", uploaded_audio_path)

"""## 15. End-to-end test: STT + CNN classification + TTS

Upload a real recording of a single supported command for meaningful results. The included synthetic test file contains no speech, so STT will not produce a useful transcript.
"""

def prepare_audio_file(path):
    y_audio, sr_audio = librosa.load(path, sr=SAMPLE_RATE, mono=True)
    if len(y_audio) < NUM_SAMPLES:
        y_audio = np.pad(y_audio, (0, NUM_SAMPLES - len(y_audio)))
    else:
        y_audio = y_audio[:NUM_SAMPLES]
    return y_audio.astype(np.float32)

if "uploaded_audio_path" not in globals():
    raise NameError("Run the upload cell immediately above this cell first.")

if not os.path.isfile(uploaded_audio_path):
    raise FileNotFoundError(
        f"Could not find {uploaded_audio_path}. Please run the upload cell again."
    )

demo_wave = prepare_audio_file(uploaded_audio_path)
print("Audio loaded:", uploaded_audio_path, "| samples:", len(demo_wave))

# STT is independent of the CNN classifier.
transcript = ""
if stt_pipe is not None:
    try:
        stt_result = stt_pipe(uploaded_audio_path)
        transcript = stt_result.get("text", "").strip()
    except Exception as e:
        print("STT failed:", repr(e))
else:
```

```
    print("STT unavailable: Whisper was not initialized.")

print("STT transcript:", transcript if transcript else "(No speech recognized)")

# CNN classification
demo_features = audio_to_logmel(demo_wave)[np.newaxis, ..., np.newaxis]
demo_probabilities = model.predict(demo_features, verbose=0)[0]
pred_idx = int(np.argmax(demo_probabilities))
predicted_command = str(label_encoder.classes_[pred_idx])
confidence = float(demo_probabilities[pred_idx])

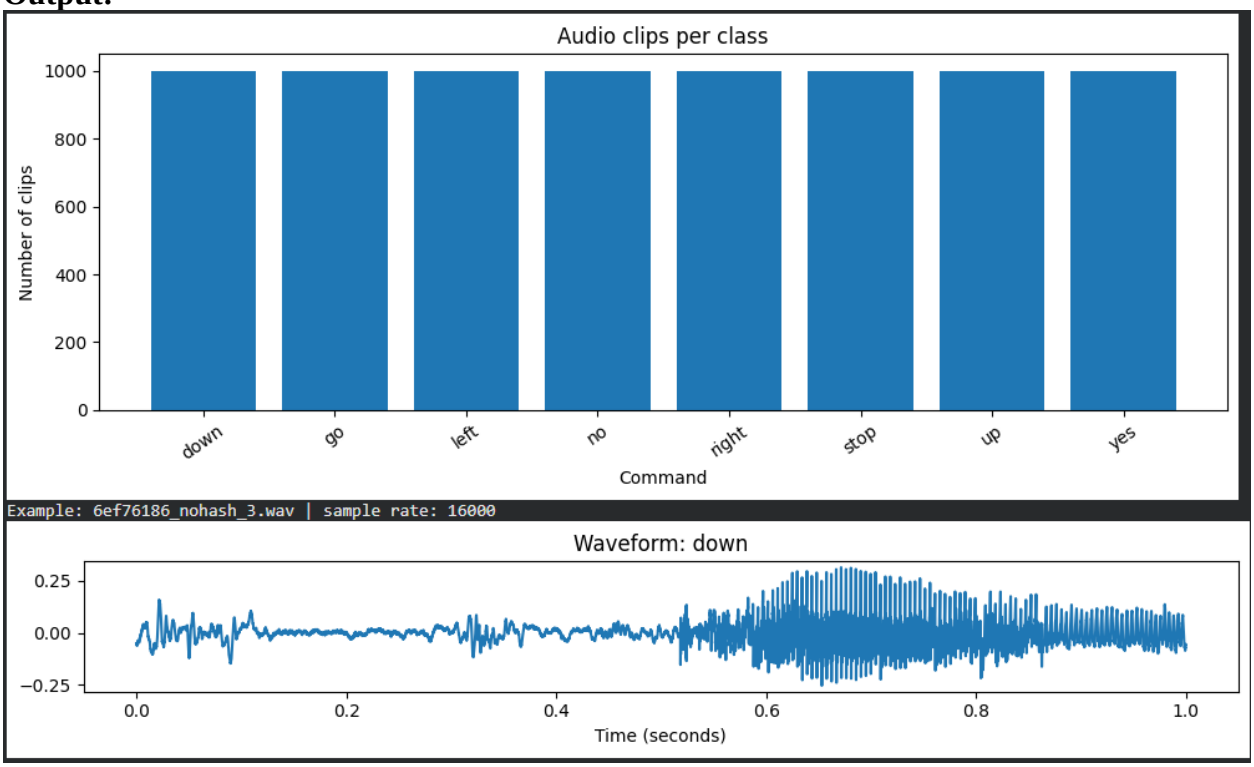
print("\nPredicted command:", predicted_command)
print(f"Model confidence: {confidence:.2%}")
print("\nTop 3 predictions:")
for idx in np.argsort(demo_probabilities)[::-1][:3]:
    print(f"{label_encoder.classes_[idx]}: {demo_probabilities[idx]:.2%}")

# TTS errors do not erase STT or classification results.
try:
    speak_text(f"The predicted command is {predicted_command}.")
except Exception as e:
    print("TTS failed:", repr(e))

"""## 16. Export the evaluation metrics"""

results = pd.DataFrame([{
    "Model": "CNN on log-Mel spectrograms",
    "Dataset": "Mini Speech Commands",
    "Test Accuracy": test_accuracy,
    "Weighted Precision": precision,
    "Weighted Recall": recall,
    "Weighted F1-score": f1,
    "Training Samples": len(X_train),
    "Validation Samples": len(X_val),
    "Test Samples": len(X_test)
}])
display(results.round(4))
results.to_csv("audio_classification_metrics.csv", index=False)
print("Saved audio_classification_metrics.csv")
```

**Output:**


Audio clips per class
Number of clips
1000
800
600
400
200
0
down
go
left
no
right
stop
up
yes
Command
Example: 6ef76186_nohash_3.wav | sample rate: 16000
Waveform: down
0.25
0.00
−0.25
0.0
0.2
0.4
0.6
0.8
1.0
Time (seconds)

Model: "sequential"

| Layer (type) | Output Shape | Param # |
|---|---|---|
| conv2d (Conv2D) | (None, 64, 63, 32) | 320 |
| batch_normalization (BatchNormalization) | (None, 64, 63, 32) | 128 |
| max_pooling2d (MaxPooling2D) | (None, 32, 31, 32) | 0 |
| conv2d_1 (Conv2D) | (None, 32, 31, 64) | 18,496 |
| batch_normalization_1 (BatchNormalization) | (None, 32, 31, 64) | 256 |
| max_pooling2d_1 (MaxPooling2D) | (None, 16, 15, 64) | 0 |
| conv2d_2 (Conv2D) | (None, 16, 15, 128) | 73,856 |
| batch_normalization_2 (BatchNormalization) | (None, 16, 15, 128) | 512 |
| max_pooling2d_2 (MaxPooling2D) | (None, 8, 7, 128) | 0 |
| dropout (Dropout) | (None, 8, 7, 128) | 0 |
| global_average_pooling2d (GlobalAveragePooling2D) | (None, 128) | 0 |
| dense (Dense) | (None, 128) | 16,512 |
| dropout_1 (Dropout) | (None, 128) | 0 |
| dense_1 (Dense) | (None, 8) | 1,032 |

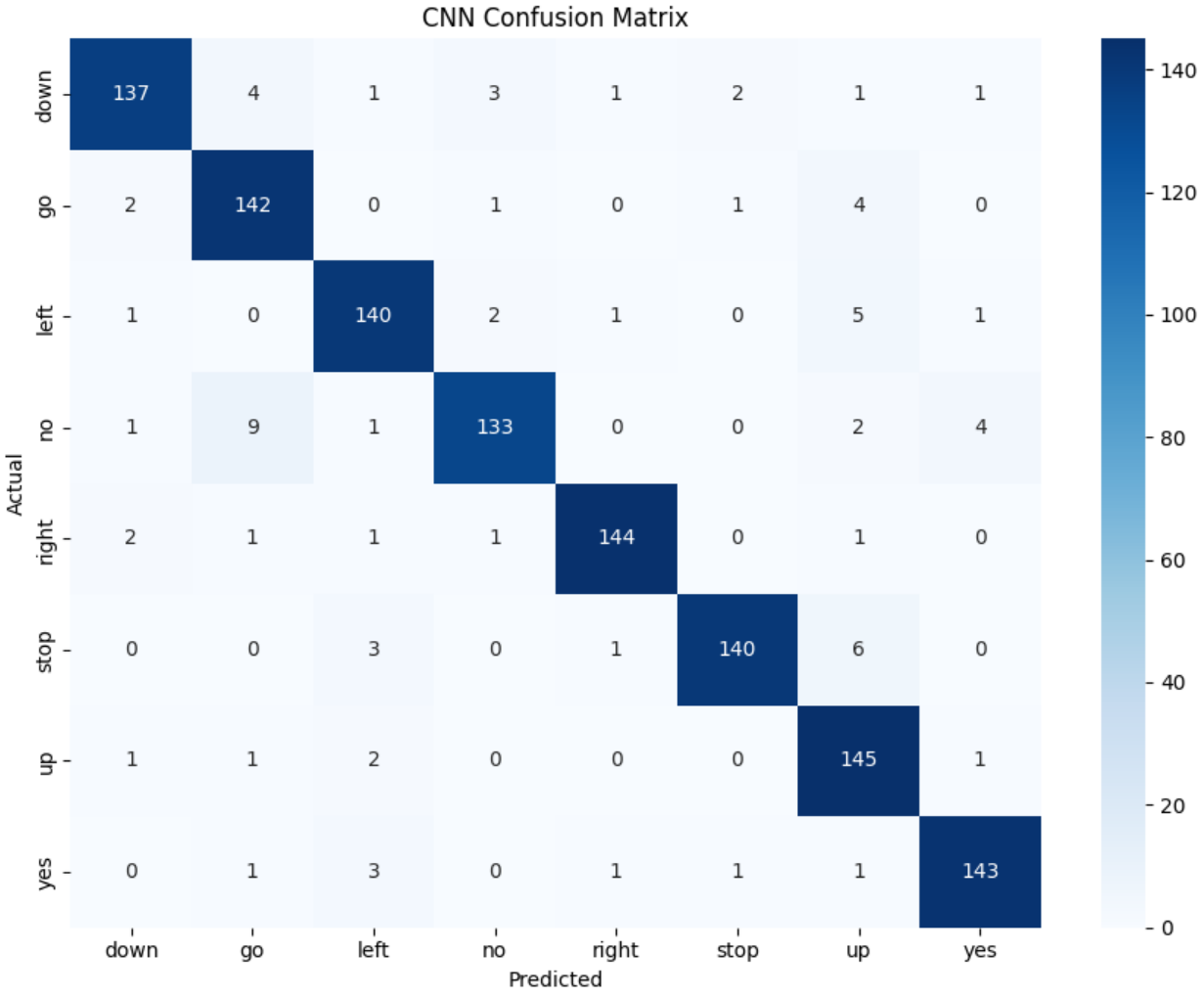
Total params: 111,112 (434.03 KB)
Trainable params: 110,664 (432.28 KB)
Non-trainable params: 448 (1.75 KB)

```
Test accuracy      : 0.9367
Weighted precision : 0.9381
Weighted recall    : 0.9367
Weighted F1-score  : 0.9368

Classification report:
              precision    recall  f1-score   support

        down       0.95      0.91      0.93       150
          go       0.90      0.95      0.92       150
        left       0.93      0.93      0.93       150
          no       0.95      0.89      0.92       150
       right       0.97      0.96      0.97       150
        stop       0.97      0.93      0.95       150
          up       0.88      0.97      0.92       150
         yes       0.95      0.95      0.95       150

    accuracy                           0.94      1200
   macro avg       0.94      0.94      0.94      1200
weighted avg       0.94      0.94      0.94      1200
```




```
Audio loaded: no.wav | samples: 16000
[transformers] Using custom `forced_decoder_ids` from the (generation) config. This is d
[transformers] Passing `generation_config` together with generation-related arguments=({
[transformers] Ignoring clean_up_tokenization_spaces=True for BPE tokenizer WhisperToken
STT transcript: No.

Predicted command: go
Model confidence: 64.08%

Top 3 predictions:
go: 64.08%
no: 35.46%
down: 0.40%
```

▶ 0:02 / 0:02

| | Model | Dataset | Test Accuracy | Weighted Precision | Weighted Recall | Weighted F1-score | Training Samples | Validation Samples | Test Samples |
|---|---|---|---|---|---|---|---|---|---|
| 0 | CNN on log-Mel spectrograms | Mini Speech Commands | 0.9367 | 0.9381 | 0.9367 | 0.9368 | 5600 | 1200 | 1200 |

Saved audio_classification_metrics.csv